

一、认真阅读文档，学习 Git 的基本用法，并回答文档中的问题。

1、你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

答：

有，打数学建模的时候和队友一起编程，之前的小组作业合作也协同开发过。当时都是通过分工的方式，每个人负责一部分代码，最后把代码统一邮件发送汇总到一个人那里，由他负责整合产出最终代码。通过本次实验，我发现了 Git 在团队协作中可以发挥重要作用，通过 Git 的分支和合并功能就可以让多人同时开发不同部分，最后统一整合，避免邮件传来传去导致版本混乱和时间耗费。

2、为什么要设计“暂存-提交”两个步骤？

答：

Git 把“暂存-提交”拆成两步，是为了让人可以精确控制每一次存档的内容，而不全存进去。如果对文件进行了多处修改，但这次只想提交其中一部分，没有暂存区，就只能“全提交”。而有了暂存区，人们就可以在改完代码后，先用 `git add` 把想要提交的内容放进暂存区，作为“预提交”的缓冲地带，所有被 `git add` 加进去的改动，才会被下一次 `git commit` 打包成一次存档。当执行 `git commit` 时，Git 默认只会把暂存区里的内容生成一条永久记录。那些没被 `git add` 的改动，会继续留在工作区，不会被这次提交带走。这样每次提交的记录都是干净、有意义的，以后回退或查历史时也能清晰识别。

3、`git branch` 和 `git branch -a` 的区别是什么？

答：

这两个命令的区别在于查看分支的范围。`git branch` 只列出本地分支，前面带 * 号的是当前所在的分支。比如本地有 `main` 和 `feature` 两个分支，它只会显示这两个。`git branch -a` 中，额外的“-a”是“--all”的缩写，表示列出所有分支，包括本地分支和远程跟踪分支，远程分支会以 `remotes/origin/分支名` 的形式显示。

二、使用此仓库建立个人仓库，完成 `main.c` 文件中的 `TODO` 部分并进行一次 `commit`。

实验步骤：

（一）创立个人仓库

进入 <https://github.com/ICS-26Fall-FDU/GitLab>，点击 Use this template, 选择 Create a new repository，将个人仓库命名为 `Tanjiayun-Lab-0`，点击 create repository

1 General

Owner *  TANJIAYUN0817 / Repository name *

✓ Your new repository will be created as Tanjiayun-Lab-0. The repository name can only contain ASCII letters, digits, and the characters ., -, and _.

Great repository names are short and memorable. How about [psychic-adventure?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Creating repository...

(二) 将仓库克隆下载到本地

1、打开本地仓库，选择 code，复制 SSH 码

 Tanjiayun-Lab-0 Public  

main 1 Branch 0 Tags

File	Commit
TANJIAYUN0817 Initial commit	Initial commit
.github/workflows	Initial commit
Makefile	Initial commit
README.md	Initial commit
main.c	Initial commit

README

Local Codespaces

Clone

HTTPS SSH GitHub CLI

git@github.com:TANJIAYUN0817/Tanjiayun-Lab-0.git

Use a password-protected SSH key.

Open in GitHub Copilot app

Open with GitHub Desktop

Download ZIP

2、打开终端，输入：

#回到家目录下的 ICS2026 文件夹（存放所有实验）

cd ~/ICS2026

#将 GitHub 上建好的仓库完整下载到本地

git clone git@github.com:TANJIAYUN0817/Tanjiayun-Lab-0.git

#进入刚刚下载好的仓库文件夹，开始修改代码

cd ~/ICS2026/Tanjiayun-Lab-0

#打开 vs code

code .

```
cdtanjiayun@localhost:~$ cd ~/ICS2026
tanjiayun@localhost:~/ICS2026$ git clone git@github.com:TANJIAYUN0817/Tanjiayun-Lab-0.git
Cloning into 'Tanjiayun-Lab-0'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
Receiving objects: 100% (8/8), done.
remote: Total 8 (delta 0), reused 7 (delta 0), pack-reused 0 (from 0)
```

```
tanjiayun@localhost:~/ICS2026/Lab0-Tanjiayun$ cd ~/ICS2026/Tanjiayun-Lab-0
tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ code .
```

(三) 修改提交 main.c

1、修改代码第五行内容，改为 `printf("I am the main branch\n");`

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("I am the main branch\n");
6      printf("Hello, world!\n");
7  }
```

2、上传提交修改

`git add .` # 暂存

`git commit -m "完成修改"` # 提交

`git push` # 把本地存档推到 GitHub 云端

```
● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git add

● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git commit -m "完成修改"
[main 3f34e3f] 完成修改
1 file changed, 1 insertion(+), 1 deletion(-)

● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 363 bytes | 181.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:TANJIAYUN0817/Tanjiayun-Lab-0.git
9b2a1db..3f34e3f  main -> main
```

三、在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。（15 分）

答：

网页 1 的阮一峰的《Commit message 和 Change log 编写指南》主要讲如何规范地写提交说明并自动生成更新日志。文章先讲了为什么要规范 Commit message，主要有以下几个好处：第一，提供更多历史信息，方便快速浏览；第二，可以过滤某些 commit，方便快速

查找；第三，可以直接从 commit 生成 Change log，即发布新版本时用来说明与上一个版本差异的文档。

接下来重点介绍了 Commit message 的格式。每次提交的说明分三个部分：Header、Body、Footer。Header 是必需的，Body 和 Footer 可以省略。Header 格式为 <type>(<scope>): <subject>，其中 type 限定七种：feat 新功能、fix 修补 bug、docs 文档、style 格式、refactor 重构、test 测试、chore 构建或辅助工具变动，scope 用于说明 commit 影响的范围，subject 不超过 50 字符，以动词开头、第一人称现在时、首字母小写、结尾不加句号；Body 写详细动机和与以前行为的对比，Footer 用于不兼容变动或关闭 Issue，撤销提交则以 revert 开头，后面跟着被撤销 Commit 的 Header，Body 固定写 This reverts commit <hash>。

最后介绍了三个配套工具的功能以及如何安装，包括 Commitizen（用 git cz 代替 git commit 交互式生成规范提交）、validate-commit-msg（通过 Git hook 检查提交信息是否合格）和 conventional-changelog（根据规范自动生成 CHANGELOG.md，分 New features、Bug fixes、Breaking changes 三部分）。

网页 2《Gitflow 使用规范》则讲团队协作中如何用分支管理代码，Gitflow 是一套让不同分支各司其职、避免多人协作时代码冲突和版本混乱的策略。

它定义了五类关键分支。master 是永久分支，与线上环境一致，要求最高稳定性，只用于存放所有发布，每次有新 commit 后立即打 tag。develop 也是永久分支，是开发主分支，从 master 分离，不直接用于开发 feature，新功能要单独创建 feature 分支，下一版本所有功能开发完后，从 develop 创建 release 分支，release 合并到 master 后再合并回 develop，合并用 --no-ff，然后删掉 release。feature 是临时分支，一个新功能对应一个，从 develop 创建，完成后合并回 develop 并删除。release 也叫 bugfix 分支，是临时分支，用于修复测试环境 bug，建议一个 bug 单独切一个分支，从 develop 创建后更新版本号并单独做 commit，修完合并到 master 和 develop 并删除。hotfix 也是临时分支，用于紧急修复线上 bug，从 master 或出问题的 tag 创建，修完合并到 master 和 develop 并删除。整个流程遵循“从哪里来，最后回到哪里去”的原则，合并时用 --no-ff 保留分支历史。操作上既可以手动执行 Git 命令，也可以用 git-flow 扩展库简化。

文章还讲了版本号规范，格式是 0.0.0 ~ 999.999.999，分别对应重大版本、迭代版本和热修复版本，master 上打 tag 不带 v，只用数字和点；分支命名推荐用短横线，也可按团队习惯用下划线。最后列了参考文档。

结合这两份材料，我认为学习 Git 是非常有必要的。Git 是现代软件协作的基础设施，没有它，多人同时改同一份代码几乎必然混乱，而 Gitflow 这样的分支策略让并行开发、按需合并成为可能。同时，Git 让代码历史成为可读的文档，规范的 Commit message 能让提交历史回答“这个功能什么时候加的”“为什么这么改”“哪个版本引入了 bug”，而不是一堆“修改”“提交”之类的无意义记录。Git 还支持自动化与工程化，Change log 自动生成、提交信息校验、CI/CD 与 release 分支配合，都说明它不只是存档工具，而是可以嵌入整个研发流程的基础设施。对个人而言，Git 是能力的放大器：可以随时回退到任意版本，可以开实验分支大胆试错而不影响主线，还可以把项目托管到 GitHub 参与全球协作。更重要的是，Git 培养了版本思维和协作纪律，让每次变更都需要有明确目的、明确范围、明确归宿，这种思维不仅适用于代码，也适用于任何需要多人协作、持续迭代的工作。所以学习 Git，既是学习如何使用一个协作工具，更是学如何有条理地管理变化、追溯历史。

四、学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交(10 分)。随后将 feature 分支 merge 到 main 分支(即切换回 main 分支执行 git merge feature)，并处理发生的合并冲突(10 分)。

实验步骤:

(一) 新建 feature 分支

在终端输入 git switch -c feature

```
● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git switch -c feature
Switched to a new branch 'feature'
```

(二) 在 feature 分支修改 main.c 并提交

1、将第五行 printf("I am the main branch\n");修改为 printf("I am the feature branch\n");

2、在第七行新增 printf("I like the feature branch\n");

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("I am the feature branch\n");
6      printf("Hello, world!\n");
7      printf("I like the feature branch\n");
8  }
```

3、暂存提交

git add . # 暂存

git commit -m "修改了 feature" # 提交

```
● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git switch -c feature
Switched to a new branch 'feature'
● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git add .
● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git commit -m "修改了 feature"
[feature 4cdec5f] 修改了 feature
1 file changed, 2 insertions(+), 1 deletion(-)
```

(三) 在 main 上修改 main.c 提交后合并

1、切回 main 分支，在终端输入 git switch main

```
● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

2、第 5 行不变，在第 7 行增加 printf("I like the main\n");

```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("I am the main branch\n");
6      printf("Hello, world!\n");
7      printf("I like the main\n");
8  }

```

3、暂存提交

git add . # 暂存

git commit -m "修改了 main" # 提交

```

● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git add .
● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git commit -m "修改了main"
[main e912c71] 修改了main
1 file changed, 1 insertion(+)

```

4、执行合并，在终端输入 git merge feature

(四) 触发冲突并解决

1、报错

```

⊗ tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.

```

```

1  #include <stdio.h>
2
3  ✓ int main()
4  {
5      printf("I am the feature branch\n");
6      printf("Hello, world!\n");
7      采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
      <<<<<< HEAD (当前更改)
      printf("I like the main\n");
9      =====
10     printf("I like the feature branch\n");
11     >>>>>> feature (传入的更改)
12 }

```

2、手动修改 main.c，删除冲突标记，保留了最终想要的代码

```

1  #include <stdio.h>
2
3  ✓ int main()
4  {
5      printf("I am the feature branch\n");
6      printf("Hello, world!\n");
7      printf("I like the main\n");
8      printf("I like the feature branch\n");
9  }

```

3、重新暂存提交

git add . # 暂存

git commit -m "解决了合并冲突" # 提交

```

tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git add .
tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git commit -m "解决了合并冲突"
[main 383975f] 解决了合并冲突

```

(五) 上传

1、将 main.c 上传到 GitHub，在终端输入 git push

2、将分支上传到 GitHub，在终端输入 git push -u origin feature

```
tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git push
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 865 bytes | 288.00 KiB/s,
done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (
from 0)
remote: Resolving deltas: 100% (6/6), completed with 1
local object.
To github.com:TANJIAYUN0817/Tanjiayun-Lab-0.git
3f34e3f..383975f  main -> main
```

```
tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-0$ git push
h -u origin feature
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (
from 0)
remote:
remote: Create a pull request for 'feature' on GitHub
by visiting:
remote:      https://github.com/TANJIAYUN0817/Tanjiayu
n-Lab-0/pull/new/feature
remote:
To github.com:TANJIAYUN0817/Tanjiayun-Lab-0.git
* [new branch]      feature -> feature
branch 'feature' set up to track 'origin/feature'.
```

思考:

实验中,在第五行对 feature 做出改动,main 保持不变,合并时第五行直接被修改为 feature 中改动过的内容,在在第七行对 main 和 feature 中都添加了不同的内容,合并时报错。

git merge 冲突的本质是,同一个位置两个分支进行了不同的改动,Git 无法判断该保留谁。第五行只有分支动过、main 没动,Git 发现此处只有一边有变化,直接采用变化的那边,不冲突;而第七行 main 和分支都加了不同的内容,Git 面对同一处有两个互相矛盾的版本无法判断,因此报冲突。进一步说,如果 main 和分支改的是不同文件、或同一文件的不同位置,Git 也能自动合并。

所以 git merge 是否可以合并成功判断标准可以概括为,改动是否落在同一处且内容不同——是则冲突,否则自动合并。